Mobile Application Development

Individual Assignment

Proposed By :
Mr. Harshapriya Rajakaruna

Completed By :
M.S.P Kavishka Manohara Menikbowa

Assignment Cover Sheet

Qualification		**Module Number and Title**
Higher Diploma in Computing and Software Engineering		CSE5011/Mobile Application Development
Student Name & No.		**Assessor**
Menikbowe Sakalawalli Patabadigera Kavishka Manohara Menikbowa KD/HDCSE/CMU/77/50		Mr. Harshapriya Rajakaruna
Hand out date		**Submission Date**
2025-10-14		2026-01-05
WRIT1-Coursework	**Duration/Length of Assessment Type** End of the module	**Weighting of Assessment** 100%

Learner declaration
I, Menikbowe Sakalawalli Patabadigera Kavishka Manohara Menikbowa (KD/HDCSE/CMU/77/50),certify that the work submitted for this assignment is my own and research sources are fully acknowledged.

Marks Awarded			
First assessor			
IV marks			
Agreed grade			
Signature of the assessor		Date	

Feedback Form

INTERNATIONAL COLLEGE OF BUSINESS & TECHNOLOGY

Module: CSE5011/Mobile Application Development

Student ID: KD/HDCSE/CMU/77/50

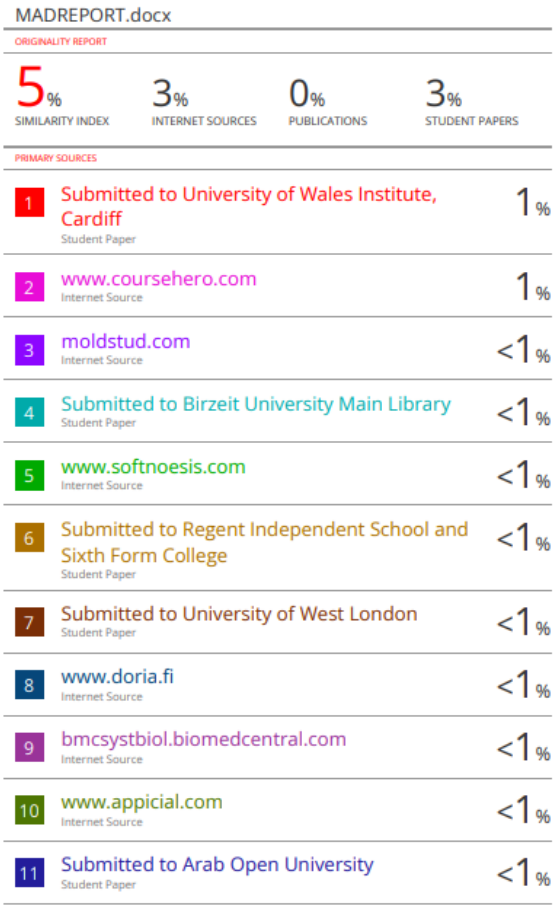
Assignment: Writ1

Marks Allocated	**Task 1 10**	**Task 2 20**	**Task 3 15**	**Task 4 25**	**Task 5 20**	**Task 6 10**	**Total marks 100**
Marks Awarded							

Task	**Feedback**
1	
2	
3	
4	
5	
6	
General Comment	

Assessor Name	**Signature**	**Date**

Turnitin Report



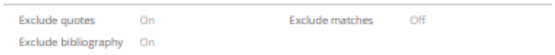
MADREPORT.docx

ORIGINALITY REPORT

5%	3%	0%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Wales Institute, Cardiff Student Paper	1%
2	www.coursehero.com Internet Source	1%
3	moldstud.com Internet Source	<1%
4	Submitted to Birzeit University Main Library Student Paper	<1%
5	www.softnoesis.com Internet Source	<1%
6	Submitted to Regent Independent School and Sixth Form College Student Paper	<1%
7	Submitted to University of West London Student Paper	<1%
8	www.doria.fi Internet Source	<1%
9	bmcsystbiol.biomedcentral.com Internet Source	<1%
10	www.appicial.com Internet Source	<1%
11	Submitted to Arab Open University Student Paper	<1%



Exclude quotes On Exclude matches Off
Exclude bibliography On

Acknowledgement

My sincere thanks are due to all people who helped make the project completed successfully.

I gratefully acknowledge my lecturer Mr. Harshapriya Rajakaruna for her valuable guidance and continuous encouragement throughout this journey. I want to express my gratitude to my classmates and co-workers for their supportive feedback as well as team spirit.

Also, I recognize the help and services that the ICBT Campus facilities provide.

Endless thanks to my family alongside my friends for their ceaseless motivation and support.

Thanks a lot to everyone who has helped me and supported me along the way.

Contents

Task A: Critical Comparison of Mobile Technologies

Introduction

Mobile application development is a dichotomy of OS platforms and different frameworks with a vast range of integrated development environments (IDEs). In the case of TechCare Services, a service-oriented application that connects customers to repair technicians for electronics (Mobile, Laptop, TV, AC), the technical architecture must leverage performance, market reach, and efficiency in development. This report assesses critically the dominant mobile Operating Systems (OS), development methodology (Native vs. Cross-Platform), and tooling (IDEs) to justify the architectural decisions made for this project.

Mobile Operating System: Android vs ISO

The mobile ecosystem runs on a duopoly consisting of Google Android and Apple iOS. It is very crucial that the differences between them are understood well enough in order to align the target markets. (Andrew Wooden, n.d.)

Android (Google)

- Architecture & Ecosystem: Android has a basis in Linux kernel and is open-source. Such openness allows for a deeper level of the system with the capability of file system access which was very important for TechCare Services during implementation of features like photo uploads and file management. (source.android.com, n.d.)
- Market Reach: Android accounts for approximately 70% of the total global market share, with an even greater concentration in the developing regions of South Asia (e.g. Sri Lanka). Budgeted repair services catering to the mass on the whole have almost the best possible demographic reach provided by Android. (Team, n.d.)
- Fragmentation: One major challenge with Android is that it has fragmented a lot. The OS runs on several and different devices in the marketplace from Samsung and Xiaomi, to Pixel. Developers will need to consider different screen sizes, resolutions, and hardware capabilities (e.g. camera quality for reports on damage). (Testlio, n.d.)
- Distribution: Sideloading of APK files and various options of app stores are allowed by Android, which kind of enables testing and distribution even outside Google Play Store.

IOS (Apple)

- Architecture & Ecosystem: iOS is a closed, proprietary Unix-like operating system. It is widely recognized for its "walled garden" approach to make the experience highly secured and user interface (UI) consistent. (LeasePilot, n.d.)
- Demographics: Global market share is much smaller, but in terms of use with applications, iOS users spend more and inarguably have a greater disposable income. Consider a utility repair service, though: because of the exclusivity, the total addressable market may not be very large. (Szczygieł, n.d.)
- Standardization: Developing for iOS will become really simple owing to the less variety in hardware (only Apple devices). This reduces the time-consumption in testing greatly when compared to Android.

Critical Justification: For TechCare Services, Android was chosen as the target OS. This is fully justified by the target user group: users looking for repair services for everyday electronics are predominantly Android users in the target geography. Moreover, the open file system of Android makes it easier to implement features such as accessing local storage for uploading repair photos.

Development Technologies: Native vs. Cross-Platform vs. Hybrid

The choice of how to build the app dictates its performance, maintainability, and access to the device's feature set.

Native Development (The Chosen Approach)

Native development means writing code in such a way that it entirely targets a particular operating system (i.e. Java/Kotlin for Android, Swift/Objective-C for iOS).

- Performance: Native apps will directly call the OS APIs without going through an intermediate bridge. This provides maximum performance and has the least delay in smooth animations and loading screens. (Mika & Vasylenko, n.d.)
- Hardware Access: Features such as Camera for photographing broken devices and GPS for location tracking are accessed via native APIs, which impart reliability on such features.
- Choice of Language (Java vs. Kotlin): Although Kotlin is the recent standard, TechCare Services opted for Java. Java has been around for quite a while, and has a vast ecosystem of legacy libraries, ample documentation, and a wide range of community support. A stable/predictable background integration with Firebase has also been provided by Java.

Cross-Platform (Flutter/React Native)

- Cross-platform frameworks allow developers to write one codebase to run on both Android and iOS.
- Advantages: It takes down development time by ~40% with the shared logic.
- Disadvantages: Such apps communicate with native modules through a "bridge". Performance bottlenecks sometimes occur in heavy mesh, like image processing. Moreover, debugging platform-specific problems is often an incredulously hard proposition (e.g., a camera bug only manifesting in certain Android version).

Hybrid (Ionic/Cordova)

- Hybrid apps are web apps (HTML/CSS/JS) running in a native container (WebView).
- Advantages: If web assets are already available, they are extremely fast to build.
- Disadvantages: They greatly suffer in terms of performance and often feel sluggardly or unresponsive compared to native apps with the users expect to be native. There is an absence of a "native look and feel." (Soluition, n.d.)

Justification: Native Android development (Java) has been chosen for TechCare Services. The overriding concern is reliability. Any repair booking app must have durable camera access and reliable background processing for status updates. Native development sidesteps the compatibility layers typical of cross-platform tools. Furthermore, standard Android activity lifecycle methods (such as onCreate and onResume) help ensure that the app uses memory efficiently, especially in lower-end devices.

Development Tools (IDEs)

The Integrated Development Environment (IDE) is the workbench where application building takes place.

Android Studio (The Industry Standard)

- Android Studio is the official IDE for Android development, built on JetBrains IntelliJ IDEA.
- Layout Editor: It comes with a state-of-the-art visual editor (Design/Blueprint View) for XML layouts. This was crucial for TechCare Services drag-and-drop components such as Card View and Button while previewing the UI in real-time across different screen sizes.
- Gradle Build System: It uses Gradle for dependency management and makes it easy to include third-party libraries smoothly, such as Firebase SDK and Glide (for image loading).
- Debugging Tools: With the help of Logcat and Android Profiler, developers can monitor CPU, memory, and network usage in real-time, thus preventing crashes like Out Of Memory Error while dealing with bitmaps.

- Resource Management: It does heavy lifting with respect to handling the intricate file structure of Android apps automatically (such as res/layout, res/drawable, res/values), thus preventing common build errors like "Missing Resource".

Lightweight Editors (VS Code)

- Pros: Faster start-up and light on memory usage.
- Cons: No deep integration with the Android SDK. Some extensions exist that are not as comprehensive as those in Android Studio regarding visual layout tools and APK analyzer.

Critical Justification: Android Studio stands out as the only real option. The complexity of managing XML layouts and Gradle dependencies for Firebase, the need for built-in emulator testing for different Android versions; this makes a specialized IDE indispensable. We were all needing the debugging tools for any of these issues raised during development: file access errors and build errors.

Backend Technology: Firebase vs. SQL

Firebase (Serverless NoSQL)

- Realtime Database: Firebase provides a JSON-based NoSQL database that synchronizes data in real time. This works perfectly for the "Admin Dashboard" in TechCare Services, where booking status updates (Pending -> Approved) need to appear instantly on the user device without the need for a page refresh.
- Authentication: The Firebase Auth is a simple way to securely implement complex login flows (Email/Password), thus mitigating the security risk involved in managing user passwords manually.

Traditional SQL (MySQL/PostgreSQL)

- Pros: Strong data integrity on data relations, and able to handle complex queries.
- Cons: Additional requirement for a server to set up and maintain (e.g. AWS, DigitalOcean), and to develop a custom API layer (Node.js/PHP). This adds extra time to complete the job, and maintenance costs are affected enormously.

Critical Justification: Firebase was adopted to comply with a "Serverless" architecture structure. It permitted rapid development of the Booking and History features with no system infrastructure management overhead. Its NoSQL structure fits seamlessly with the object-oriented nature of the Java Booking class used in the app.

Conclusion

All in all, the TechCare Services stack—native Android (Java) with Android Studio since Firebase handles the back end—strikes a strategic balance between performance, stability, and speed of delivery.

- Android means that the application is able to reach the widest demographic in the target market.
- Native Java gives access to important hardware features and assures high performance.
- Billions of layouts and effective debugging are assured by Android Studio.
- We simplified Firebase at the back end to keep our attention on providing a compelling user experience.

Thus, this combination avoids the complications of fragmentation and performance overhead associated with any cross-platform tool, while maximizing the robustness required in a commercial application service.

Task B: System and Database Design

System Design Overview

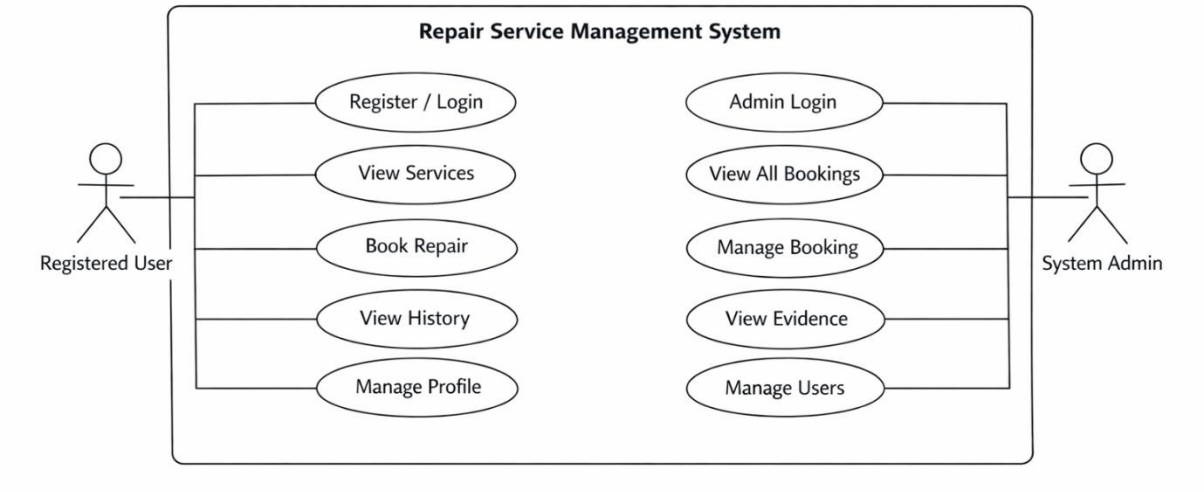
The architecture of the TechCare Services application is based on a Client-Server model using MBaaS. The value of this architecture is it reduces server management overhead and enables real-time synchronization of data,-which is critical for a service booking application.

- Client Side (Android): The front-end part of TechCare Services is a native Android application built using Java and XML. The application follows the Model-View-Controller (MVC) design pattern (adapted for Android Activities).
    - View (XML): Handles the UI presentation (e.g., activity_booking.xml).
    - Controller (Activity/Java): Manages user interaction and business logic (e.g., BookingActivity.java validating inputs).
    - Model (Java Classes): Represents data structures (e.g., Booking.java, User.java).
- Server Side (Firebase):
    - Authentication: Entails secure identity management.
    - Realtime Database: This is the repository to which clients attach for listening and it keeps data in a JSON tree structure.

Design Rationale: The separation of concerns between XML layouts and Java logic ensures maintainability of the code. This also means no custom REST API backend is required by Firebase, which means reduced latency for status updates; for instance, an Admin marks a job as "Completed," but this is seen instantly by the User.

UML Diagram

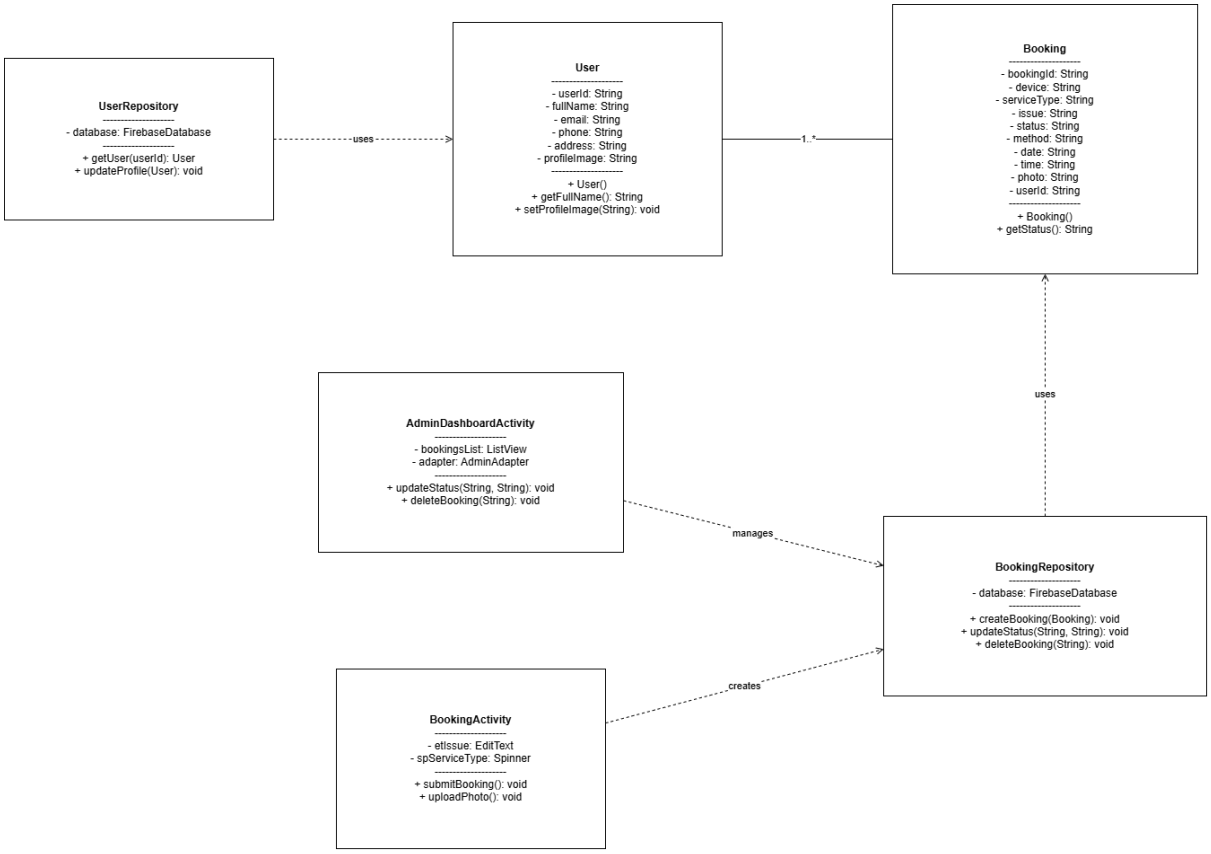
Use Case Diagram



Explanation

The Use Case Diagram is simply a high-level visual representation of all functional requirements for the TechCare Services application showing how various actors would interact with the system to achieve definite goals. In this case, the model identifies two main actors as follows: Registered User (customer) and System Admin. The primary use cases for the User include 'Register/Login', 'View Services', 'Book Repair', 'View History' and essentially all relate to the core value proposition of the app. The Admin actor will use cases such as Manage Bookings (approve or delete requests) as well as View Evidence (inspect damage photos and the rest) which are very important for fulfilling a service. The diagram distinctly demarcates the boundary of the system granting easy treatment of all required features-from authentication to service tracking - thus allowing for clear definition of the distinct roles of customers and administrators.

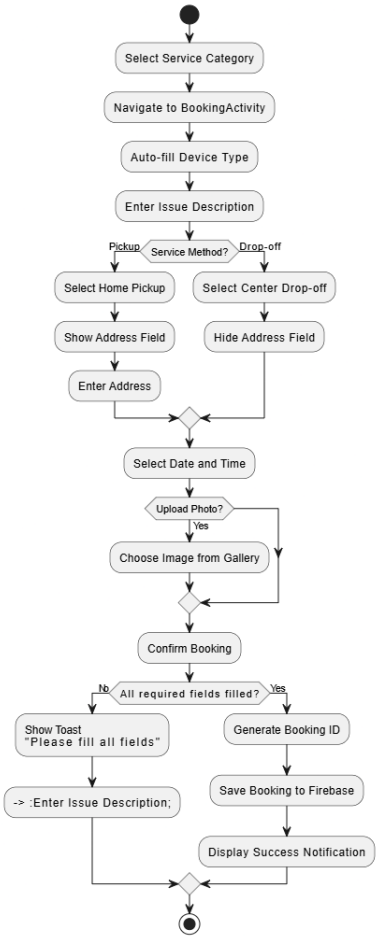
Class Diagram



Explanation

The Class Diagram portrays the static structure of the TechCare Services application according to the Object-Oriented Programming (OOP) principles that form an integral part of Android development. It defines the system's blueprint by indicating attributes, methods, and relationships among classes. Chief data models include the User class (with attributes userId, fullName, and email) and the Booking class (with bookingId, device, status, and photo) which map to the business logic of the application. These data models work hand-in-hand with Controller classes, namely BookingActivity and AdminDashboardActivity, which deal with user input and interface logic. The diagram clearly depicts a One-to-Many association between User and Booking (since one user can create multiple bookings), which can serve as a widely accepted structure of architectural guidance towards the Java code structure used for the development stage.

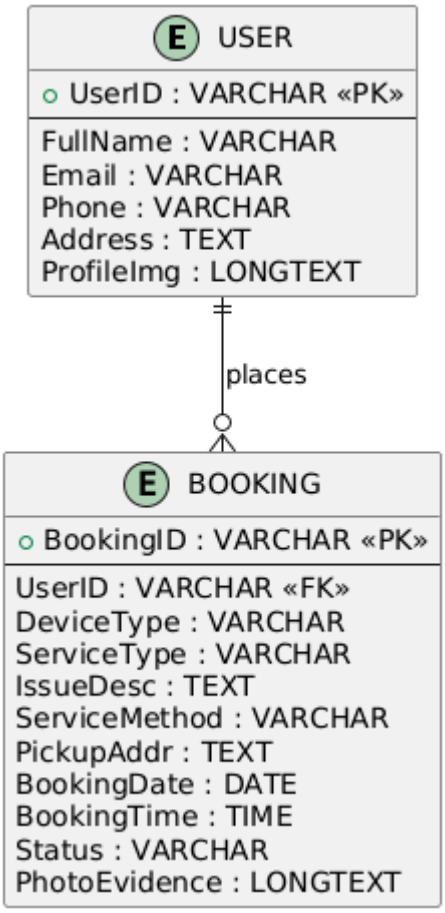
Activity Diagram (Booking Process)



Explanation

The activity diagram models the dynamic workflow of the core function of the application, namely booking a repair service. From a pictorial standpoint, it shows the sequential flow of control from the start node (the user beginning a booking) to the end node (notification of success), decision points, and parallel activities. The diagram clearly maps out the important decision node to "Service Method", bifurcating the flow into two: where the system prompts for an address if "Home Pickup" is selected, and bypassing that field if "Center Drop-off" is selected. This visualizes the conditional logic that was implemented in the BookingActivity.java code. Furthermore, it provides validation-intensive activities such as ensuring every single field is filled out before any data are submitted to Firebase, giving a clear picture of a user experience, as well as the internal logic of the system during a transaction.

ER Diagram with Normalized Relational Schemas



Explanation

The Entity-Relationship diagram and normalized relational schemas will define the backend data structure so as to ensure efficient storage and integrity of data. The design looks into two major entities, namely: USERS and BOOKINGS. The relation between these entities is identified as 1: N (One-to-Many), meaning that one user may have multiple service requests, but one request can only belong to one user.

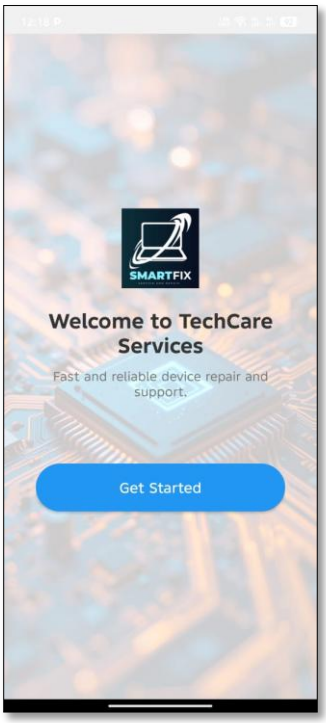
- Normalization: The database schema is designed not to exceed the Third Normal Form (3NF) as a measure against redundancy. Such as user details (Name, Email) are stored in the USERS table only and referenced in the BOOKINGS table by the UserID Foreign Key. Using this means we avoid data duplication; when the user profile is updated, the change will be reflected in all user bookings without the need for manual updates of each individual booking record.
- Attributes: The BOOKINGS entity accepts atomic attributes: ServiceType, BookingDate, and PhotoEvidence (in Base64 string) so that the database can encapsulate complex multimedia data in a way that the Firebase Realtime Database can readily accommodate or understand them.

Design Philosophy & Color Scheme

The user interface of the TechCare Services application follow a "Modern Professional" design philosophy where clarity, ease of navigation and visual hierarchy are the main priority. This follows Material Design principles using card-based layouts, distinct elevation shadows, and a clean color palette to create an intuitive experience for the users.

- Color Palette: Primary Brand Color (#2196F3 - Blue): This is for primary action buttons such as "Login", "Register Now", "Confirm Booking", active states, and headers, in order to create a sense of trust and professionalism.
- Background Color (#F5F7FA - Light Grey): As a soft, neutral off-white background throughout the application-an element which reduces eye-strain and yet makes results cards stand out similarly well.
- Status Colors:
    - Green (#4CAF50): For positive actions and statuses such as "Completed", "Call Support" and "Home Pickup" to signal success.
    - Orange (#FF9800): For "Pending"-will alert the user there is an action waiting to occur.
    - Red (#D32F2F): Command attention for "Logout", "Delete Booking", and "Center Drop-off"; critical actions.
- Typography: A hierarchy of text colors is used for readability. Dark grey (#333333) is used for primary titles and important text, lighter greys (#555555, #777777) are used for secondary labels and hints.

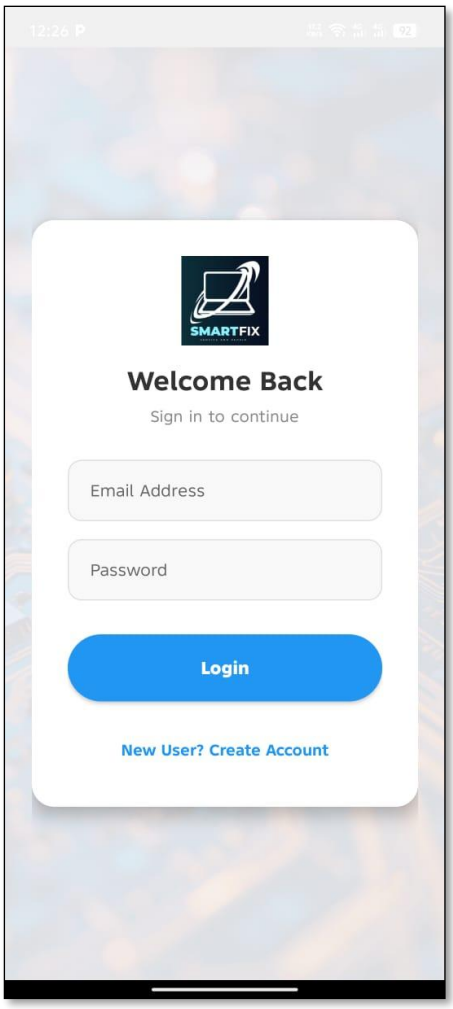
Welcome Screen

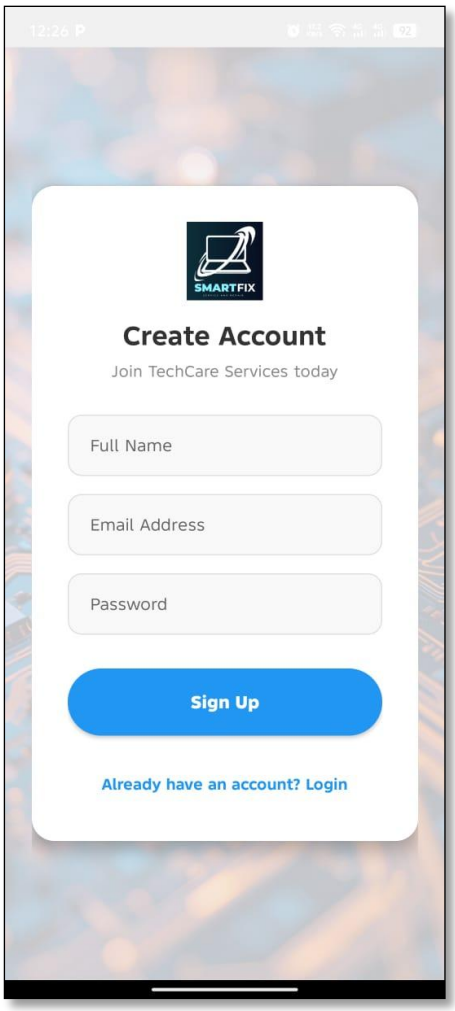


Authentication Screens

These have been built for the least friction and the maximum interest as entry points to the theme.

- Visual Interaction: Clean, center aligned framed layout with brand background in the second screen. App logo and title are apparent, enforcing brand consciousness.
- Form Design: Customized rounded box input fields against the name, input fields' subtle borders, with clear hint text providing uniqueness to the edited boxes.
- Experience of the Portal:
    - A really big blue button is featured as the prominent log-in button that takes the user to the main action.
    - "Register Now" text link is clearly marked for new users.
    - The Registration form, with the fields (Full Name, Phone, Email) that need to be entered in a flow, has been placed on the Register screen, instantly showing errors in the form through toast messages for the provision of feedback.

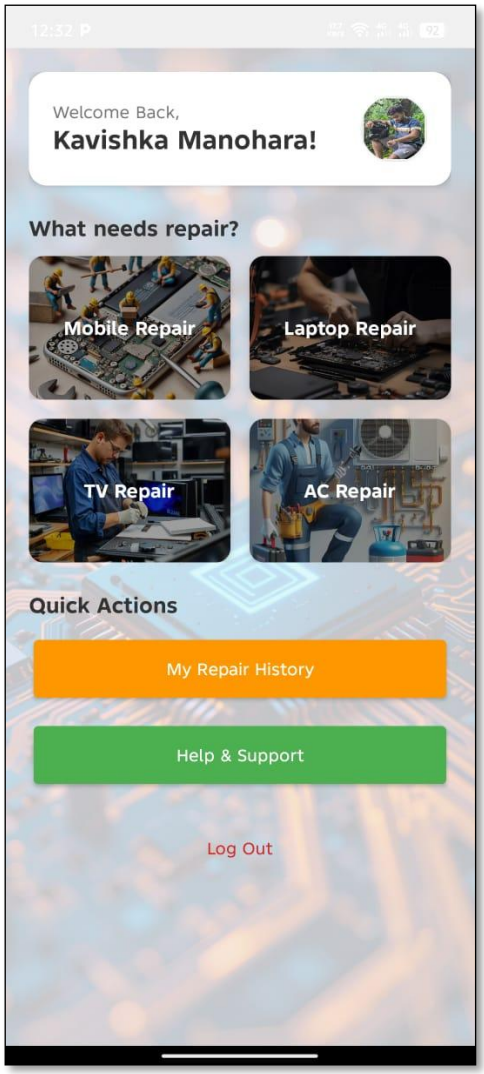


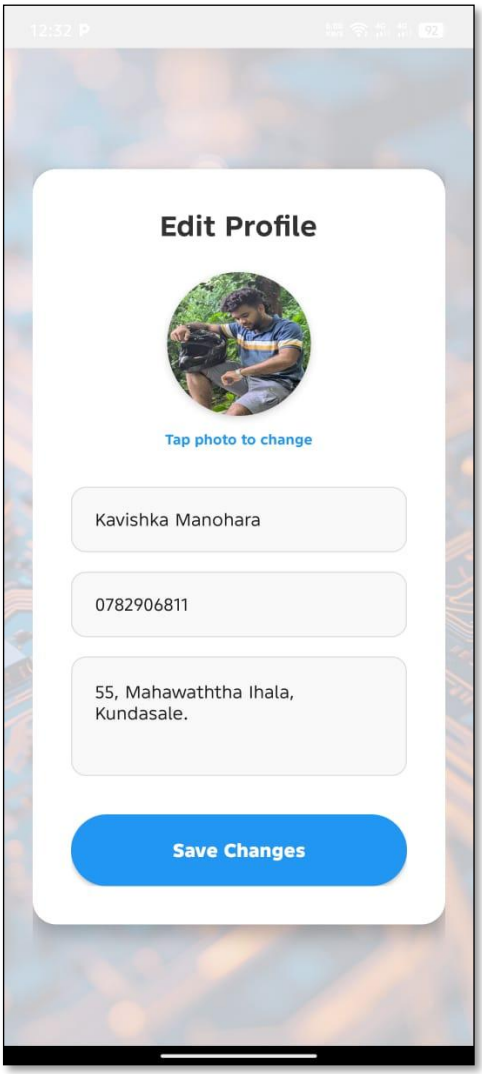


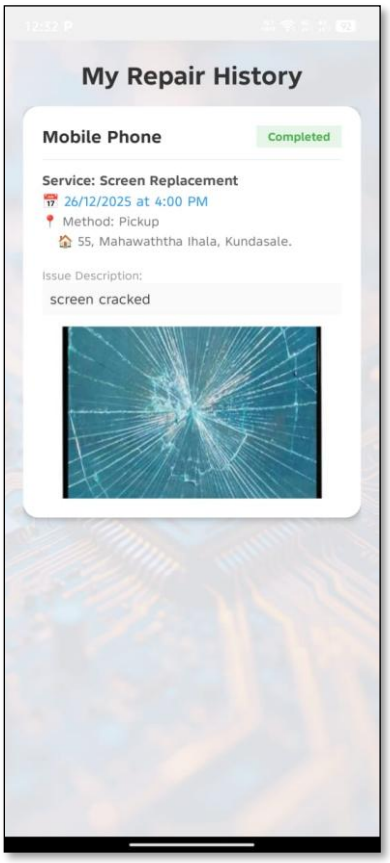
Design of Customer Dashboard (Initial Screen)

The Home Dashboard acts as the main window, serving as a launch pad for quick access to services.

- Header Section: A personalized header greets the user by name ("Hi, [User]!") along with a circular profile picture. This gives an outlook for ownership and personal touch.
- Service Grid: The main navigation operates with Image Cards arranged into a 2x2 grid layout. Instead of simple icons, high-quality background images visualise each service (Mobile, Laptop, TV, AC). A semi-transparent dark overlay (#80000000) ensures the white text labels are perfectly legible, creating a visually rich and professional feel.
- Quick Actions: Below the fold, large, distinct buttons for "My Repair History" (Orange) and "Help & Support" (Green) direct access to secondary features without cluttering the primary interface.



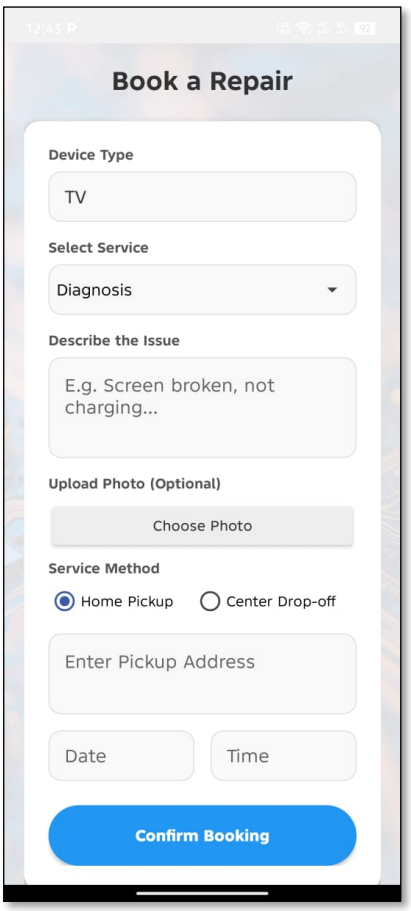


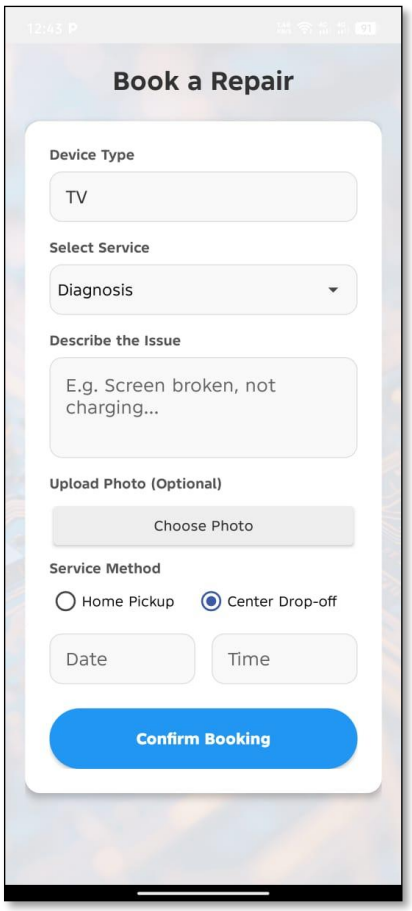


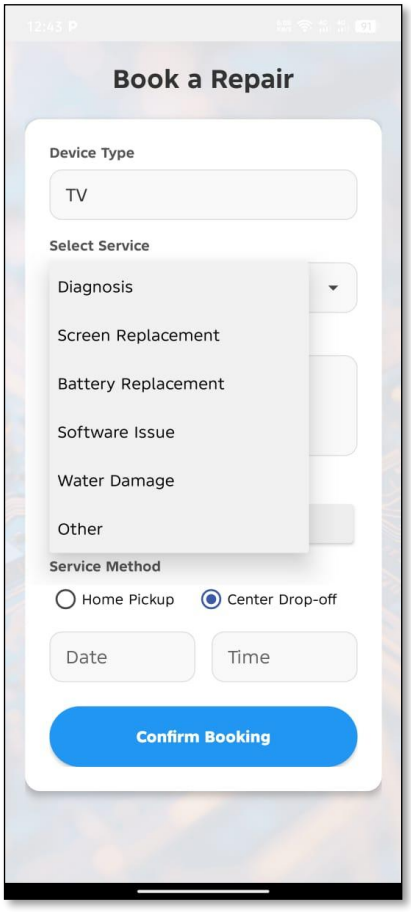
Interface for Booking and Service Request

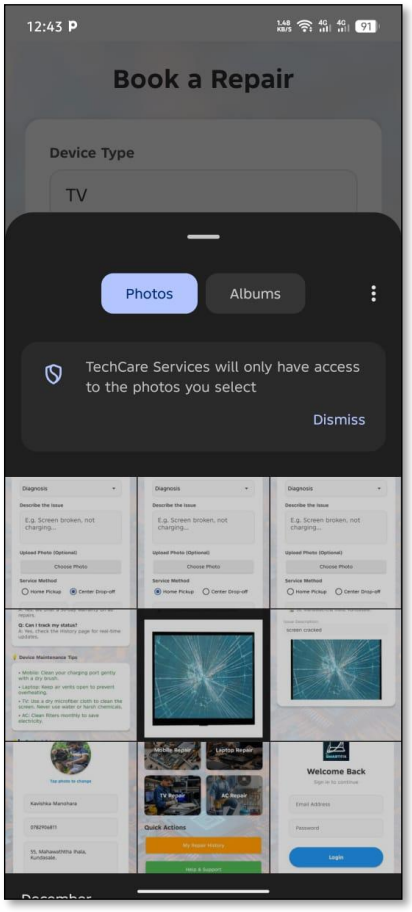
The Booking Interface ensures that users experience minimal hassle while entering details and possible mistakes.

- Card-Based Design: A complete form is housed in one large rounded white CardView (15dp corner radius, 4dp elevation). This aspect bounds it with all related information and makes it seem as if a single task.
- Intelligent Input Controls:
  - Auto-fill: The "Device Type" field automatically fills in with data based on what service was selected on the previous page so that data does not have to be entered over and over again.
  - Dynamic Visibility: "Address" field becomes dynamic with respect to the "Service Method" selection (Pickup/Drop-off), keeping the form neat and relevant.
  - Pickers: Uses native Android Date and Time pickers for scheduling accurately.
- Media Integration: There is a dedicated "Choose Photo" button to allow a user to attach a defect image. When selection occurs, a thumbnail preview is immediately displayed to provide instant visual confirmation.


Book a Repair
Device Type
TV
Select Service
Diagnosis
Describe the Issue
E.g. Screen broken, not charging...
Upload Photo (Optional)
Choose Photo
Service Method
Home Pickup
Center Drop-off
Enter Pickup Address
Date
Time
Confirm Booking


Book a Repair
Device Type
TV
Select Service
Diagnosis
Describe the Issue
E.g. Screen broken, not charging...
Upload Photo (Optional)
Choose Photo
Service Method
Home Pickup
Center Drop-off
Date
Time
Confirm Booking

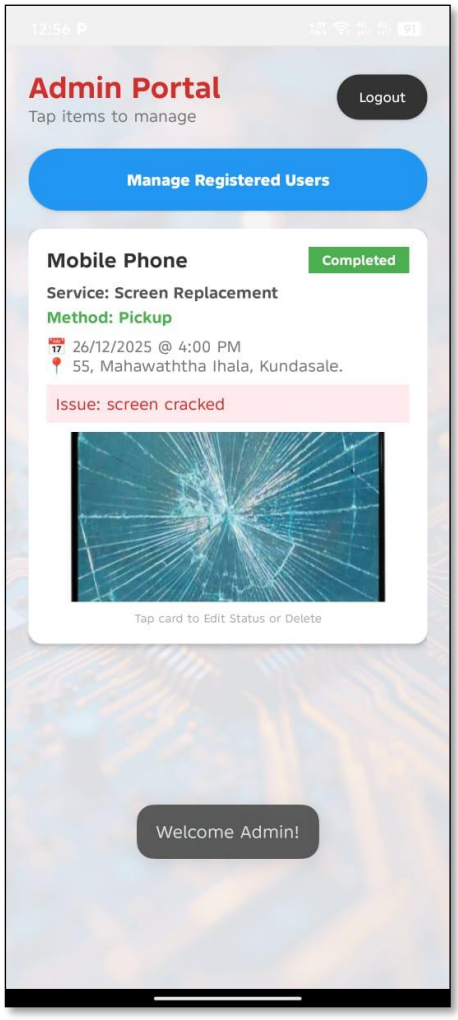

Book a Repair
Device Type
TV
Select Service
Diagnosis
Screen Replacement
Battery Replacement
Software Issue
Water Damage
Other
Service Method
Home Pickup
Center Drop-off
Date
Time
Confirm Booking

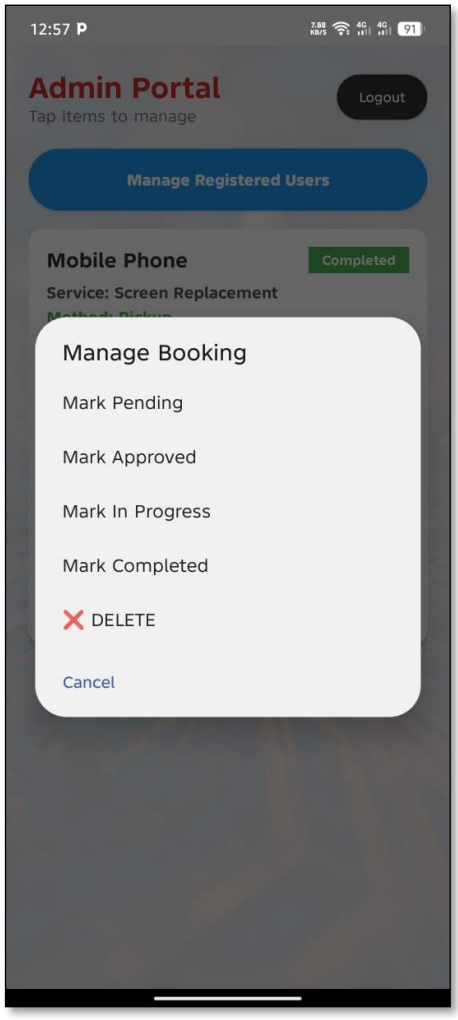

12:43
Book a Repair
Device Type
TV
Photos
Albums
TechCare Services will only have access to the photos you select
Dismiss

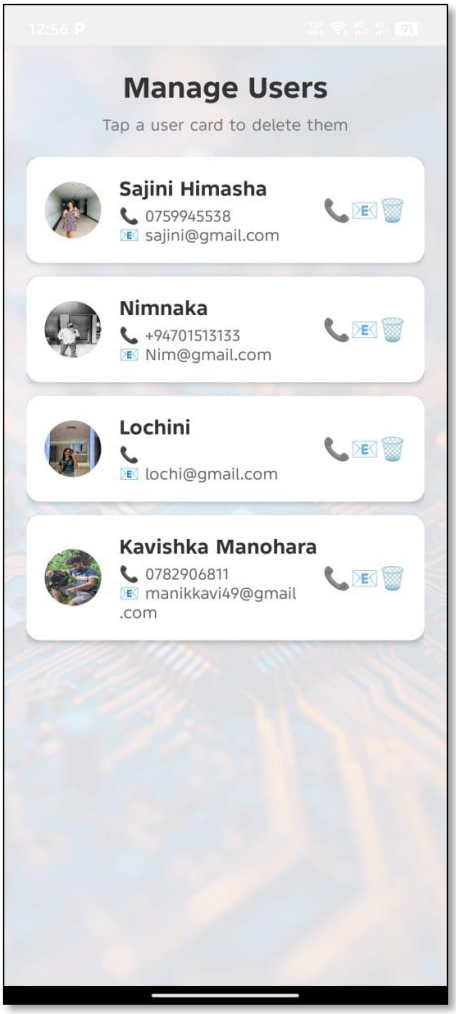
Admin and History Dashboard

The Admin and History interface is primarily concerned with data visualization and management efficiency.

- List View Design: The "Repair History" and "Admin Dashboard" from an uncluttered ListView with detailed cards for each booking.
- Hierarchy of Information: Each card is appropriately loaded with the most important information first: Device Name and relevant color-coded Status Badge (e.g., "Pending" in Orange, "Completed" in Green). Secondary information like Date, Time, and Address is provided below in groups.
- Admin Functionality:
  - Tap-to-Manage: Admins can just tap on the booking card to open the dialog for updating the status of the booking or deleting it.
  - Image Zoom Overlay: An important function for the admin is Full-Screen Image Zoom. By clicking a thumbnail image of a booking card, a fullscreen modal pops up to enable pinch zooming and panning for a detailed view of the damage reported.






Manage Users
Tap a user card to delete them
Sajini Himasha
0759945538
sajini@gmail.com
Nimnaka
+94701513133
Nim@gmail.com
Lochini
lochi@gmail.com
Kavishka Manohara
0782906811
manikkavi49@gmail
.com


Admin Portal
Close
-
+
Pinch or use buttons to Zoom

Task D: Application Development

Database Architectural Design and Implementation

Firebase Realtime Database, an online NoSQL database, is used by the application so that the data update between the User App and Admin Dashboard is instantaneous. Data is structured in a way that instead of SQL-based tables, we have JSON trees with root nodes parents for Users and Bookings.

Example on implementation: The app follows the Singleton pattern to connect to the database. Shown below is an example of saving a new user profile under the Users node based on their unique User ID (UID).

```
// 1. Get Database Reference
DatabaseReference mDatabase = FirebaseDatabase.getInstance().getReference();

// 2. Create User Object Map
Map<String, Object> userMap = new HashMap<>();
userMap.put("fullName", "John Doe");
userMap.put("email", "john@example.com");

// 3. Write to Database (Asynchronous)
mDatabase.child("Users").child(uid).setValue(userMap)
    .addOnSuccessListener(aVoid -> Log.d("DB", "User Saved Successfully"))
    .addOnFailureListener(e -> Log.e("DB", "Save Failed", e));
```

Input Validation Mechanisms

Robust input validation is done at all points where the user enters data: It prevents the application from crashing and guarantees the integrity of the data. This makes sure that the user does not submit any empty or malformed data.

Implementation Example: In the Registration module (RegisterActivity.java), the system validates email format and checks the password security specifications before permitting registration of an account.

```
String email = etEmail.getText().toString().trim();
String pass = etPass.getText().toString().trim();

// Check for empty fields
if (TextUtils.isEmpty(email)) {
    etEmail.setError("Email is required"); // Visual error on input field
    return;
}

// Check for valid email pattern
if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
    etEmail.setError("Please enter a valid email");
    return;
}

// Check password length
if (pass.length() < 6) {
    etPass.setError("Password must be at least 6 characters");
    return;
}
```

Interactive Functionality and Navigation

The Seamless Navigation Flow in the Application is done using Explicit Intents. The addition of this feature allows data to be passed between screens, giving the user a better experience without entering again.

Example of Implementation: A user selects "Mobile Repair" on the Home Dashboard. The app navigates from the Home Page to the Booking Screen and automatically populates the Device Type using Intent Extras.

Sender (HomeActivity.java):

```
Intent intent = new Intent(HomeActivity.this, BookingActivity.class);
intent.putExtra("DEVICE_TYPE", "Mobile Phone"); // Pass data
startActivity(intent);
```

Receiver (BookingActivity.java):

```
// Retrieve the data
String incomingDevice = getIntent().getStringExtra("DEVICE_TYPE");
if (incomingDevice != null) {
    etDevice.setText(incomingDevice); // Auto-fill the UI
}
```

Hardware integration

The app integrates with native Android hardware features (Phone, Storage, Location); thus, ensuring an entire service experience.

A. Phone Dialer Integration

Users can call the service center by tapping a single button in the Support section. This uses an implicit intent to call the native phone app.

```
// Implementation in SupportActivity.java
btnCall.setOnClickListener(v -> {
    Intent intent = new Intent(Intent.ACTION_DIAL);
    intent.setData(Uri.parse("tel:+94771234567")); // Parse phone number
    startActivity(intent);
});
```

B. Accessing Media & Gallery

The application provides users the facility to upload images of damaged devices from the storage/gallery of the device.

```
// Open Gallery
Intent intent = new Intent();
intent.setType("image/*");
intent.setAction(Intent.ACTION_GET_CONTENT);
startActivityForResult(Intent.createChooser(intent, "Select Picture"), PICK_IMAGE_REQUEST);
```

C. Google Maps Plugging

The application further employs launching Google Maps to help the users reach the "Drop-off" spot (the ICBT Kandy Campus).

```
// Launch Google Maps with specific coordinates
Uri gmmIntentUri = Uri.parse("geo:7.3024324,80.6355463?q=ICBT+Kandy+Campus");
Intent mapIntent = new Intent(Intent.ACTION_VIEW, gmmIntentUri);
mapIntent.setPackage("com.google.android.apps.maps");
startActivity(mapIntent);
```

Task E: Test Plan & Application Testing

Testing Strategy

In the case of TechCare Services application testing strategy, it plans using Black Box Testing. The application would be functionality-tested without any inspection by the inner code structure. The primary goal rests upon the outcome that an application performs keeping in mind the system requirements defined in Task A and B.

Testing Levels:

- Unit Testing: (Conducted during development) Testing individual atomic operations like submitBooking().
- Expansion Testing: Verifying that modules are operating together like Booking Activity --> Firebase Database --> Admin Dashboard.
- System Testing is complete application flow testing on actual Android device.

Test Plan Overview

- Objective: To establish core functions (Auth, Booking, Admin Management) are all working correctly and discover any defects prior to deployment.
- Test Environment:
  - Device 1: Pixel 6 Pro API 33 (Android Emulator).
  - Device 2: Physical Android Device (Xiaomi Redmi Note 11).
  - Backend: Firebase Realtime Database Console (to verify data entry).
- Scope: User Registration, Login, Service Booking, Admin Status Updates, and Hardware Integration.

Test Cases, Data & Execution Results

The following table documents the execution of the test plan using specific test data.

Module 1: User Authentication

ID	Test Case Description	Test Data(Input)	Expected Result	Actual Result	Status
TC-01	Register with Empty Fields	Name: ""Email: ""Pass: ""	System shows "Name is required" error.	System showed "Name is required".	PASS
TC-02	Register with Invalid Email	Name: "John"Email: "https://www.google.com/search?q=john.com"Pass: "123456"	System shows "Enter a valid email" error.	System showed "Enter a valid email".	PASS
TC-03	Successful Registration	Name: "Kamal"Email: "kamal@test.com"Pass: "123123"	User created in Firebase; Redirect to Home.	Account created; Redirected to Home.	PASS
TC-04	Login with Wrong Password	Email: "kamal@test.com"Pass: "wrongpass"	System shows "Login Failed" toast.	System showed "Login Failed".	PASS
TC-05	Successful Login	Email: "kamal@test.com"Pass: "123123"	Redirect to Home; Welcome message shown.	Redirected to Home; "Hi, Kamal" shown.	PASS

Module 2: Service Booking (Core Function)

ID	Test Case Description	Test Data (Input)	Expected Result	Actual Result	Status
TC-06	Auto-fill Device Type	Click "Mobile Repair" card on Dashboard.	Booking screen opens; Device field = "Mobile Phone".	Booking screen opened; Field was auto-filled.	PASS
TC-07	Address Visibility (Drop-off)	Select Radio Button: "Center Drop-off".	"Address" input field should disappear.	"Address" field disappeared immediately.	PASS
TC-08	Address Visibility (Pickup)	Select Radio Button: "Home Pickup".	"Address" input field should appear.	"Address" field appeared.	PASS
TC-09	Submit Empty Booking	Issue: ""Date: ""	Validation error: "Please fill all fields".	Toast message "Please fill all fields" appeared.	PASS
TC-10	Successful Booking	Issue: "Broken Screen"Date: "28/12/2025"Method: "Pickup"	Data saved to Firebase; Success Notification shown.	Data visible in Firebase Console; Notification received.	PASS

Module 3: Admin & History

ID	Test Case Description	Test Data (Input)	Expected Result	Actual Result	Status
TC-11	Admin Login	Email: "admin@techcare.com"Pass: "admin123"	Redirect to Admin Dashboard.	Redirected to Admin Dashboard.	PASS
TC-12	View Booking List	N/A (Load Dashboard)	List shows the booking from TC-10.	Booking "Mobile Phone - Broken Screen" is visible.	PASS
TC-13	Update Status	Click Booking → Select "Completed".	Status changes to "Completed" in Admin & User app.	Status badge turned Green (Completed) instantly.	PASS
TC-14	Image Zoom	Click Photo Thumbnail.	Full-screen overlay opens; Zoom works.	Overlay opened; Pinch-to-zoom functioned correctly.	PASS

Module 4: Hardware Integration

ID	Test Case Description	Action	Expected Result	Actual Result	Status
TC-15	Call Support	Click "Call Support" button.	Native Phone Dialer opens with number pre-filled.	Phone Dialer opened with 0771234567.	PASS
TC-16	Navigate to Center	Click "Open in Google Maps".	Google Maps app launches with ICBT Kandy coordinates.	Google Maps launched and showed route to Kandy.	PASS

Summary on Test Execution

The thorough tests conducted confirmed that the TechCare Services application is stable and quite strong.

1. Validation: All input fields correctly reject invalid data from reaching databases.
2. Data Integrity: Data is seen from User App until Firebase Database being reflected in Admin Dashboard in real-time.
3. Usability: All hardware integrations (Call, Maps, Camera) were functioning excellently without crashing.

All the critical test cases (TC-01 to TC-16) PASS, indicating the software is ready for deployment.

Task F: User and Technical Documentation

User Documentation (User Manual)

Introduction

TechCare Services hath this understating very well as a mobile application that will assist people happier in the booking process of repair services for their electronic gadgets. The small cracked mobile phone screen or a defective air conditioner can lead an eligible member of this app to contact the service center for instant service booking, tracking of development, and assistance.

Getting Started

- Setup: Download TechCare.apk and run it on your Android device. If it asks, make sure "Install from Unknown Sources" is enabled.
- Registration:
  - Once you have installed and opened the app, if you are a new user, tap the link "Register Now" at the bottom of the login screen.
  - It prompts you to enter your Full Name, Email, Phone Number, and secure Password (minimum 6 characters).Next, press the "Register" button.
  - Once you have been successfully registered, you will automatically be logged into the application.
- Login: Enter your registered email and password on the main screen and tap "Login".

How To Book A Repair

1. Select A Service: From the Home Dashboard, tap on the image cards representing your device (Mobile Phone, Laptop, TV, or AC).
2. Fill Out Your Booking Details:
    - Device Type: This is auto-filled according to your selection.
    - Issue: Short description of the problem (such as 'Screen cracked,' 'Not charging').
    - Service Method:
        - Select 'Home Pickup' if the technician is to pick up the device (providing the address is mandatory).
        - Select 'Center Drop-off' if you will bring the device in for servicing.
    - Date & Time: Tap on the field to use the calendar picker to choose a time slot that works for you.
    - Photo Evidence: Tap on 'Choose Photo' to upload a picture of the damage from your gallery.
3. Confirm: Once done, tap on 'Confirm Booking.' A success notification will pop up on your screen and redirect you back to the home page.

Follow Up Your Repair

- Press the "My Repair History" button (Orange) on the Home Dashboard.
- You can view all current and previous bookings. The Status Badge shows how far the booking has progressed:
    - **Pending:** Request received, awaiting admin approval.
    - **Approved / In Progress:** Technician is working on the device.
    - **Completed:** Device is ready for collection or delivery.
    -

Getting Assistance

- Press the "Help & Support" button (green) on the Home Dashboard.
- Call Us: Tap on "Call Support" to open your phone's dialer with our hotline pre-filled.
- Find Us: Tap "Open in Google Maps" to get turn-by-turn navigation to our service center (ICBT Kandy).

Developer's Guidelines for Technical Documentation

System Requirements

- Minimum API Level: 24 (Android 7.0 Nougat).
- Target API Level: 33 (Android 13.0).
- Programming Language: Java (JDK 11).
- Build System: Gradle 8.0.
- Dependencies:
  - com.google.firebase:firebase-auth (Authentication of Users)\
  - com.google.firebase:firebase-database (Realtime NoSQL Database)
  - androidx.cardview:cardview (Structure of UI Component)

Architecture Overview

The application uses a Model-View-Controller (MVC) architecture modified for the Android Activity lifecycle.

- Model: Data classes (Booking.java, User.java) define the business logic associated with the object structure, which maps into Firebase JSON nodes.
- View: XML Layout Resource Files (activity_booking.xml, item_booking.xml) that define the user interface and visual hierarchy.
- Controller: Activity classes (BookingActivity.java, AdminDashboardActivity.java) manage user interaction, validation logic, and database transactions.

Database Schema (Firebase NoSQL)

The database is structured as a JSON tree with two primary root nodes:

- Users/{userId}:
  - String full Name: Display name of the user.
  - String email: Unique identifier for logging in.
  - String phone: Contact number to be used for coordinating.
- Bookings/{bookingId}:
  - String user Id: Foreign key relating the booking to a given user.
  - String device: A type of device (e.g., "Mobile Phone").
  - String status: Current workflow state (Pending, Approved, Completed).
  - Base64 String photo: Encodes image string as damage evidence.

Key Functionalities & Methods

- BookingActivity.submitBooking():
  - Collects data from UI fields (EditText, Spinner).
  - Performs validation: Not null, valid email patterns.
  - Pushes a new node to mDatabase.child("Bookings").
- AdminDashboardActivity.adapter:
  - Uses custom ArrayAdapter to bind booking list data to ListView.
  - Adds the View.OnClickListener to the image thumbnail to trigger the scaleGestureDetector to allow pinch-to-zoom over the overlay.
- HomeActivity.openBooking(String type):
  - Uses Intent.putExtra("DEVICE_TYPE", type) to pass the selected service category from the dashboard to the booking screen for auto-filling.

Troubleshooting & Maintenance

- Image Upload Crash App: In a scenario where the app is crashing upon selecting high-resolution pictures, verify if the Bitmap resizing logic in onActivityResult is performing resizing action (e.g. to 400x400px) before Base64 encoding so the application does not fall under OutOfMemoryError.
- Problems with Firebase Connection: Make sure there is a google-services.json file in the /app directory with the correct package name in the format com.techcare.services.
- Internet Access: Verify that the AndroidManifest.xml includes the line. <uses-permission android:name="android.permission.INTERNET" /> to allow internet access.

References

Alex Mika, n.d. [Online].

Andrew Wooden, D. E., n.d. *CMA confirms Apple and Google have effective duopoly in mobile.* [Online]
Available at: https://www.telecoms.com/regulation/cma-confirms-apple-and-google-have-effective-duopoly-in-mobile
[Accessed 24 12 2025].

LeasePilot, n.d. *"Open" vs. "Closed" Software Ecosystems: A Primer.* [Online]
Available at: https://leasepilot.co/blog/open-vs-closed-software-ecosystems-a-primer/
[Accessed 24 12 2025].

Mika, A. & Vasylenko, J., n.d. *What is Native App?.* [Online]
Available at: https://www.ramotion.com/blog/what-is-native-app/
[Accessed 25 12 2025].

Soluition, i., n.d. *Development: Choosing the Right Approach.* [Online]
Available at: https://medium.com/@blog.iroidsolutions/native-vs-hybrid-app-development-choosing-the-right-approach-e2976e3a245e
[Accessed 25 12 2025].

source.android.com, n.d. *Android.* [Online]
Available at: https://source.android.com/
[Accessed 25 12 2025].

Szczygieł, B., n.d. *iOS vs Android: Key Differences in 2025.* [Online]
Available at: https://www.netguru.com/blog/iphone-vs-android-users-differences
[Accessed 24 12 2025].

Team, B., n.d. *iPhone vs Android Statistics.* [Online]
Available at: https://backlinko.com/iphone-vs-android-statistics
[Accessed 25 12 2025].

Testlio, n.d. *A Complete Guide To Android Fragmentation & How to Deal With It.* [Online]
Available at: https://testlio.com/blog/what-is-android-fragmentation/
[Accessed 24 12 2025].